Ingegneria del Software 18/06/2022 Soluzione del Tema

Soluzione del Tema

Esercizio 1

Si consideri la classe Java `HotelDB` per gestire la prenotazione di stanze di hotel.

```
public class HotelDB {
  // Ritorna l'elenco di tutti gli hotel contenuti.
  public /*@ pure @*/ Set<Hotel> getHotels();

  // Ritorna la lista di tutte le stanze che (i) possono ospitare almeno people persone,
  // (ii) si trovano vicino a location, (iii) sono libere il giorno day.
  // La lista e' in ordine crescente di prezzo (dalla stanza meno costosa alla piu' costosa).
  public /*@ pure @*/ List<Room> findRooms(int people, Location location, Day day);

  // Prenota la stanza room per il giorno day.
  // Solleva una AlreadyBookedException se la stanza non e' disponibile il giorno specificato.
  public void book(Room room, Day day) throws AlreadyBookedException;
}

public class Hotel {
  // Ritorna tutte le stanze presenti nell'hotel.
  public /*@ pure @*/ Set<Room> getRooms();

  // Ritorna true sse l'hotel e' vicino a location.
  public /*@ pure @*/ boolean isCloseTo(Location location);
}

public class Room {
  // Ritorna true sse la stanza e' disponibile il giorno day.
  public /*@ pure @*/ boolean isAvailable(Day day);

  // Ritorna il numero massimo di persone che la stanza puo' ospitare.
  public /*@ pure @*/ int getMaxPeople();

  // Ritorna il prezzo della stanza.
  public /*@ pure @*/ float getPrice();

  // Permette di prenotare la stanza nel giorno precisato.
  // Richiede che la stanza non sia gia' prenotata per lo stesso giorno.
  public void book(Day day);
}
```

Domanda a)

Si specifichi in JML il metodo `findRooms()`. **Soluzione**

```
//@ requires people > 0 && location != null && day != null
//@
//@ ensures \result != null &&
//@ (\forall Room r; r != null; \result.contains(r) <==> (
//@    (\exists Hotel h; getHotels().contains(h) && h.isCloseTo(location);
//@       h.getRooms().contains(r) ) &&
//@    r.isAvailable(day) && r.getMaxPeople() >= people) ) &&
//@ (\forall int i; i>=1 && i<\result.size();
//@    \result.get(i-1).getPrice() <= \result.get(i).getPrice())
public /*@ pure @*/ List<Room> findRooms(int people, Location location, Day day);
```

NB: La porzione

```
(\forall Room r; r != null; \result.contains(r) <==> (\exists Hotel h;... ))
```

può essere sostituita dalla forma seguente:

```
(\forall Hotel h; ...(\forall Room r; r != null && ...; \result.contains(r) <==> (... )))
```

. Le due forme in generale non sono equivalenti, ma in questo caso particolare lo sono in quanto è evidentemente sottointeso che ogni stanza puo' essere contenuta in un solo hotel.

Domanda b)

Si specifichi in JML il metodo `book()`.

Soluzione

```
//@ requires room != null && day != null &&
//@ (\exists Hotel h; getHotels().contains(h); h.getRooms().contains(room))
//@
//@ ensures \old(room.isAvailable(day)) && !room.isAvailable(day) &&
//@ getHotels().size() == \old(getHotels().size()) &&
//@ getHotels().containsAll(\old(getHotels())) &&
//@ (\forall Day d; d != null;
//@    (\forall Room r;
//@    (\exists Hotel h; getHotels().contains(h); h.getRooms().contains(r));
//@    (d != day || r != room) ==> r.isAvailable(d) == \old(r.isAvailable(d)) ) )
//@
//@ signals (AlreadyBookedException e) && !\old(room.isAvailable(day)) &&
//@ getHotels().size() == \old(getHotels().size()) &&
//@ getHotels().containsAll(\old(getHotels())) &&
//@ (\forall Day d; d != null;
//@    (\forall Room r;
//@    (\exists Hotel h; getHotels().contains(h); h.getRooms().contains(r));
//@    r.isAvailable(d) == \old(r.isAvailable(d)) ) )
public void book(Room room, Day day);
```

Domanda c)

Si consideri un'implementazione che utilizza una `List` per memorizzare gli hotel, come mostrato di seguito. Per tale implementazione si fornisca un invariante di rappresentazione e una funzione di astrazione.

```
public class HotelDB {
   private List<Hotel> hotels;
   ...
}
```

Soluzione

L'invariante di rappresentazione richiede semplicemente che la lista non sia nulla, non contenga elementi nulli e non contenga duplicati.

```
//@ private invariant hotels != null && !hotels.contains(null) &&
//@ (\forall int i; i>=1 && i<hotels.size(); !hotels.get(i).equals(hotels.get(i-1)) )
```

La funzione di astrazione definisce `getHotels()` a partire dalla lista. Tutti gli altri metodi sono definiti a partire da `getHotels()`.

```
//@ private invariant
//@ (\forall Hotel h; h != null; hotels.contains(h) <==> getHotels().contains(h))
```

Domanda d)

Si consideri una classe `HotelDB2` che aggiunge un metodo `bookAvailableRoom()` alla classe `HotelDB`. Il metodo si comporta esattamente come `book()` ma *non* solleva alcuna eccezione nel caso in cui la stanza non sia disponibile. Può `HotelDB2` essere definito come sottoclasse di `HotelDB` in accordo con il principio di sostituzione? Motivare la propria risposta.

Soluzione

`HotelDB2` può essere definito come sottoclasse di `HotelDB` in quanto aggiunge un metodo senza modificare i metodi esistenti (e le loro pre-condizioni e post-condizioni). Il metodo aggiunto inoltre non viola alcun invariante della classe.

Esercizio 2

Si consideri la seguente versione semplificata delle classi `HotelDB` e `Hotel`.

```
class HotelDB {
  private final List<Hotel> hotels;

  public HotelDB(List<Hotel> hotels) {
    this.hotels = new ArrayList<>(hotels);
  }

  public void book(int id) {
    if (hotels.get(id).getAvailableRooms() > 0) {
      hotels.get(id).bookOneRoom();
  } }
}

class Hotel {
  private int totalRooms;
  private int availableRooms;

  public synchronized int getAvailableRooms() { return availableRooms; }

  public synchronized int getTotalRooms() { return totalRooms; }

  public synchronized void bookOneRoom() { availableRooms--; }

  public synchronized void cancelOneBooking() { availableRooms++; }
}
```

Domanda a)

L'implementazione fornita risulta correttamente sincronizzata? In caso contrario, indicare come correggere l'implementazione, favorendo soluzioni che massimizzino l'esecuzione parallela di thread che accedono ai metodi della classe `HotelDB`.

Soluzione

L'implementazione *non* risulta correttamente sincronizzata, in quanto il controllo della disponibilità di stanze e la successiva prenotazione non avvengono in maniera atomica. Di conseguenza il numero di stanze disponibili in un hotel potrebbe andare sotto 0. Per correggere, si può modificare l'implementazione come mostrato di seguito.

```
class HotelDB {
  ...
  public void book(int id) {
    Hotel h = hotels.get(id);
    synchronized(h) {
      if (h.getAvailableRooms() > 0) {
        h.bookOneRoom();
  } } }
}
```

Domanda b)

Si implementi un metodo `cancel(int id)` in `HotelDB`. Tale metodo agisce sull'hotel in posizione `id`: se per tale hotel il numero di stanze disponibili risulta inferiore al numero di stanze totali (`getAvailableRooms() < getTotalRooms()`), il metodo cancella una prenotazione dall'hotel invocando il suo metodo `cancelOneBooking()`, altrimenti non fa nulla.

Si modifichi quindi il metodo `book(int id)` facendo in modo che, in caso non ci sia disponibilità nell'hotel indicato, il metodo sospenda il chiamante in attesa di una cancellazione. Si specifichi, qualora fosse necessario, quali altri metodi richiedono modifiche.

Si favoriscano soluzioni che massimizzano l'esecuzione parallela di thread che accedono ai metodi di `HotelDB`.

Soluzione

Occorre anche modificare il metodo `book()` come mostrato di seguito, per fare in modo che invochi `notifyAll()` per risvegliare i thread in attesa.

```
class HotelDB {
  ...
  public void cancel(int id) {
    Hotel h = hotels.get(id);
    synchronized(h) {
      if (h.getAvailableRooms() < h.getTotalRooms()) {
        h.cancel();
        h.notifyAll();
      }
  } }

  public void book(int id) {
    Hotel h = hotels.get(id);
    synchronized(h) {
      while (h.getAvailableRooms() == 0) {
        h.wait();
      }
      h.bookOneRoom();
  } }
}
```

Esercizio 3

```
1  public static int foo(int n) {
2    int x = n%2;
3    int y = n%3;
4    while (n != 0) {
5      if (x*y == 2 || n < 0) return x;
6      else n--;
7    }
8    return y;
9  }
```

Si determini un insieme minimo di casi di test per ciascuno dei casi seguente di copertura.

1. Copertura delle istruzioni (statement coverage)
2. Copertura delle decisioni (edge coverage)
3. Copertura delle decisioni e delle condizioni (edge and condition coverage)

Soluzione

1. Copertura delle istruzioni (statement coverage): n=2 (per entrare nel ramo else) e n=5 (per entrare nel ramo then)
2. Copertura delle decisioni (edge coverage): come sopra.
3. Copertura delle decisioni e delle condizioni (edge and condition coverage) oltre ai casi precedenti serve n=-1 per coprire il secondo disgiunto nell'OR.

Esercizio 4

Si considerino le seguenti classi Java:

```
class Device {
  public String toString() { return "a generic device"; }
}

class Watch extends Device {
  public String toString() { return "a watch"; }
}

class Smartwatch extends Watch {
  public String toString() { return "a smartwatch"; }
}

class Person {
  protected String name;
  public Person(String name) { this.name = name; }
  public void buy(Device d, double euro) {
    System.out.println(name+" buy "+d+" for "+euro+" euros");
  }
}

class Professor extends Person {
  public Professor(String name) { super(name); }
  public void buy(Device d, int euro) {
    System.out.println("Prof. "+name+" buy "+d+" for exactly "+euro+" euros");
  }
}

class FullProfessor extends Professor {
  public FullProfessor(String name) { super(name); }
  public void buy(Smartwatch d, int euro) {
    System.out.println("Full prof."+name+" buy "+d+" for exactly "+euro+" euros");
  }
}
```

e il seguente frammento di codice le cui righe sono state numerate per riferimento:

```
1  Device d1 = null, d2 = new Device();
2  Watch w = new Smartwatch();
3  Smartwatch sw = new Smartwatch();
4  Person p = new Person("Marco");
5  Professor pp = new FullProfessor("Andrea");
6  FullProfessor fp;
7
8  d1 = sw;
9  fp = pp;
10 p.buy(d1, 15);
11 p = pp;
12 p.buy(d1, 15);
13 pp.buy(w, 15);
14 fp.buy(w, 15.0);
15 fp = new FullProfessor("Luca");
16 fp.buy(w, 15.0);
17 fp.buy(w, 15);
```

Domanda a)

Scrivere il numero di riga delle istruzioni (se ne esistono) che generano un errore in compilazione o a run-time, chiarendo in quale momento (compile-time vs. run-time) viene sollevato l'errore e spiegandone brevemente le ragioni.

Soluzione

La riga 9 non compila perchè `fp` è staticamente di tipo `FullProfessor` mentre `pp` è staticamente di tipo `Professor` e non si può assegnare una sopraclasse ad una sottoclasse. Si noti che dinamicamente `pp` sarebbe del tipo corretto (`FullProfessor`) ma il compilatore guarda solo ai tipi statici. In conseguenza dell'errore precedente non va considerata neppure la gira 14.

Domanda b)

Supponendo di eliminare le eventuali righe che generano errori indicate sopra, si scriva, per ogni riga rimanente, il valore stampato in uscita.

Soluzione

Ricordando che il compilatore guarda solo ai tipi statici e sceglie di conseguenza la "firma" del metodo da invocare, mentre il run-time cerca eventuali ridefinizioni ("override") del metodo scelto dal compilatore a partire dal tipo dinamico, si ha il seguente risultato:

Riga 10 `Marco buy a smartwatch for 15.0 euros.` `p` è staticamente di classe `Person`, classe che possiede un solo metodo `buy` che, previa conversione automatica dell'intero 15 al double 15.0, può essere invocato con i parametri passati.

Riga 12 `Andrea buy a smartwatch for 15.0 euros`. Come sopra. Infatti, la classe `FullProfessor`, tipo dinamico dell'oggetto invocato, non ridefinisce il metodo `buy(Device d, double euro)` ma aggiunge ("overload") un metodo simile con parametri di tipo diverso, e lo stesso accade alla sopraclasse `Professor`.

Riga 13 `Prof. Andrea buy a smartwatch for exactly 15 euros`. In questo caso il tipo statico di `pp` è `Professor`, il compilatore sceglie quindi il metodo `buy(Device d, int euro)` aggiunto in `Professor` e compatibile con il tipo (statico) dei parametri attuali. Dinamicamente la classe `FullProfessor`, tipo dinamico di `pp`, non ha ridefinito tale metodo ma ha aggiunto ("overload") un ulteriore metodo con primo parametro di tipo `Smartwatch` (e non `Device` come nella sopraclasse).

Riga 16 `Luca buy a smartwatch for 15.0 euros`. Il tipo statico di `fp` è `FullProfessor`, a partire da questa classe il compilatore cerca un metodo compatibile con i tipi (statici) dei parametri attuali: non può scegliere il metodo `buy(Smartwatch d, int euro)`, aggiunto in `FullProfessor`, perchè il secondo parametro attuale è un `double` che non può essere convertito a `int`; per ragioni analoghe non può scegliere neppure `buy(Device d, int euro)`, aggiunto in `Professor`; di conseguenza il compilatore sceglie `buy(Device d, double euro)` definito nella classe `Person`. Dinamicamente non ci sono ridefinizioni ("overload") di questo metodo lungo la catena di ereditarietà del tipo dinamico (`FullProfessor`), di conseguenza si chiama il metodo definito in `Person`.

Riga 17 `Prof. Luca buy a smartwatch for exactly 15 euros`. Come sopra il compilatore cerca un metodo compatibile con i tipi (statici) dei parametri attuali a partire dal tipo statico `FullProfessor`: non può scegliere il metodo `buy(Smartwatch d, int euro)`, aggiunto in `FullProfessor`, perchè il primo parametro attuale è (staticamente) di tipo `Watch`, non compatibile con `SMartWatch`; può invece scegliere il metodo `buy(Device d, int euro)` perchè entrambi i parametri attuali sono compatibili con i parametri formali. All'atto della chiamata non ci sono, nella classe `FullProfessor`, tipo dinamico di `fp`, ridefinizioni del metodo scelto (il metodo definito in `FullProfessor` è un caso di "overloading"), quindi si risale la catena di ereditarietà e si esegue il metodo della classe `Professor`.

Si noti che in tutti i ragionamenti precedenti abbiamo trascurato di dire che in fase di stampa del "device" si sceglie sempre la versione del metodo `toString` ridefinita dal tipo dinamico del parametro (in questo caso non è presente "overloading" ma solo "overriding").